



Operators

What is Operators?

Operators perform operations on data:

- Arithmetic Operators
- Comparison Operators
- Logical Operators



Arithmetic Operators

Arithmetic Operators on integer and float

- + (addition)
- - (subtraction)
- * (multiplication)
- / (division)
- ** (exponentiation)
- // (integer division)
- % (modulo): Get remaining of division



Code Sample - Arithmetic Operators on integer and float

```
: x = 10
  y = 3
  print(x + y)    # Addition
  print(x - y)    # Subtraction
  print(x * y)    # Multiplication
  print(x / y)    # Division
  print(x % y)    # Modulus
  print(x ** y)   # Exponentiation
  print(x // y)   # Floor Division
```

13

7

30

3.3333333333333335

1

1000

3

Arithmetic Operators (For string)

- + (concat)
- * (multiplication)



Arithmetic Operators (On string)

```
1: x = "python"  
   y = "programing"  
   print(x + " " + y)    # concats
```

python programing

```
1: x = "A"  
   y = 3  
   print(x * y)    # Multiplication
```

AAA

```
1: x = "A"  
   y = 3  
   print(x + y)    # Concat with numbers
```

```
-----  
TypeError                                Traceback (most recent call l  
Cell In[66], line 3  
      1 x = "A"  
      2 y = 3  
----> 3 print(x + (y))  
  
TypeError: can only concatenate str (not "int") to str
```

```
1: print(x + str(y))    # Concat with numbers
```

A3



More Operation with String!

String Methods

Python provides numerous built-in methods for string manipulation, such as:

- `upper()`
- `lower()`
- `strip()`
- `split()`
- `replace()`, and more.

Code Sample

```
# Example of string methods
message = "hello world"
print(message.upper())           # Output: HELLO WORLD
print(message.strip())          # Output: hello world
print(message.split())          # Output: ['hello', 'world']
print(message.replace('o', '0'))# Output: hell0 w0rld
```

```
HELLO WORLD
hello world
['hello', 'world']
hell0 w0rld
```

String Indexing

Extract character from String

Syntax: string[index]

```
[11]: sentence = "Python is Amazing!"  
      sentence[1] # Index begin from 0
```

```
[11]: 'y'
```

```
[12]: sentence[-1] # Negative index begin form -1
```

```
[12]: '!'
```

String Slicing

Extracting Substrings from a string using slicing notation

Syntax: string[begin:end:step]

```
[29]: numbers = "0123456789"  
      numbers[1:] # begin index start from 0
```

```
[29]: '123456789'
```

```
[30]: numbers[2:4] # begin from 1st until 3th (end is exclusive!)
```

```
[30]: '23'
```

```
[33]: numbers[2::2] # default step is 1
```

```
[33]: '2468'
```

Positive String slicing

```
[29]: numbers = "0123456789"
      numbers[1:] # begin index start from 0

[29]: '123456789'

[30]: numbers[2:4] # begin from 1st until 3th (end is exclusive!)

[30]: '23'

[33]: numbers[2::2] # default step is 1

[33]: '2468'

[31]: sentence = "Python is Amazing!"
      sentence[1:]

[31]: 'ython is Amazing!'

[32]: sentence[2:4]

[32]: 'th'

[34]: sentence[2::2]

[34]: 'to sAaig'
```

Negative String slicing

```
[58]: numbers = "0123456789"
      numbers[-8:-1] # default step is positive

[58]: '2345678'

[59]: numbers[-1:-8:-1] # negative step mean traverse backward

[59]: '9876543'

[63]: numbers[-6:] # Begin, End, Step are optional

[63]: '456789'

[64]: numbers[-6:::] # Begin, End, Step are optional

[64]: '456789'

[65]: numbers[: -6:] # Begin, End, Step are optional

[65]: '0123'
```

Comparison Operators

Comparison Operators

- == (equal to)
- != (not equal to)
- < (less than)
- > (greater than)
- <= (less than or equal to)
- >= (greater than or equal to)

Numerical Comparison

```
10 == 10
```

True

```
10 != 10
```

False

```
10 <= 10
```

True

```
10 < 10
```

False

Equal Comparison

```
'test' == 'test'
```

True

```
'10' == 10
```

False

Size Comparison (sort)

```
:'test' < 'test1'
```

True

```
:'test' > 'test1'
```

False

```
:'test2' > 'test1'
```

True

Incompatible Comparisons

Python is a **strictly typed system** for comparison; data that are inherently different cannot be compared using logical operators.

```
: # Incorrect Comparison
number = 10
text = '5'
# This will raise a TypeError
result = number > text
```

```
-----
TypeError                                Traceback (most recent call last)
Input In [1], in <cell line: 5>()
      3 text = '5'
      4 # This will raise a TypeError
----> 5 result = number > text

TypeError: '>' not supported between instances of 'int' and 'str'
```

[Fix Code](#)

● **Key takeaway:** Ensure operand types match before using logical operators.

Logical Operators

Logical Operators

- and (both conditions true)
- or (at least one condition true)
- not (inverts the truth value)



and (both conditions true)

```
: value = 11  
value >= 0 and value <=10
```

: False

```
: value = 5  
value >= 0 and value <=10
```

: True

or (at least one condition true)

```
value = 11  
value <= 10 or value >=20
```

False

```
value = 25  
value <= 10 or value >=20
```

True

```
value = 5  
value >= 0 or value <=20
```

True

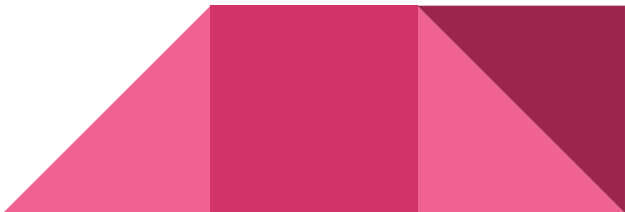
Not (inverts the truth value)

```
10 < 10
```

False

```
not(10 < 10)
```

True





TRY IT YOURSELF

Arithmetic Operators

Do math on numbers and strings: `+` `-` `*` `/` `**` `//` `%`

Arithmetic Operation

Replace each ____ with the right operator

a = 17

b = 5

print(a ____ b) # want 22 (add)

print(a ____ b) # want 12 (subtract)

print(a ____ b) # want 85 (multiply)

print(a ____ b) # want 3.4 (divide include floating)

print(a ____ b) # want 3 (divide whole number only)

print(a ____ b) # want 2 (remainder)

print(a ____ b) # want 1419857 (a power b)

Your task

- Replace every ____ with the operator that gives the wanted value.
- Mind the difference between / (float), // (whole), and % (remainder).
- Stretch: change **a** and **b**, then predict each line before running.

Division & Modulo

KNOWLEDGE CHECK

quiz1.py

```
print(17 // 5)  
print(17 % 5)
```

A. 2 and 3

B. 3.4 and 2

C. 3 and 2

D. 12 and 5

String Arithmetic

TRY IT YOURSELF · ~6 MIN

```
banner.py

# Use + to join and * to repeat
name = "Python"
line = "="

# TODO 1: make greeting read -> Hi Python!
greeting = "Hi " + name + "!"

# TODO 2: make edge a row of 20 equals signs
edge = line * 20

print(edge)
print(greeting)
print(edge)
```

Your task

- Use `+` to join strings together and `*` to repeat one.
- TODO 1 builds the greeting; **TODO 2** builds the 20-character edge line.
- Run it — the output should be a banner box around **Hi Python!**



Insight: `+` joins string-to-string only; adding a number needs `str(...)` first.

Expected output

```
=====
Hi Python!
=====
```


String Methods

Transform text with `upper()` `lower()` `strip()` `split()` `replace()`

Clean the Data

TRY IT YOURSELF · ~8 MIN

clean_data.py

```
raw = "  Ada, Grace, Alan  "

# Fill each ___ with a string method
step1 = raw.__( )          # trim outer spaces
step2 = step1.__( )        # make it UPPERCASE
names = step2.__(", ")     # split into a list of 3
fixed = raw.__( "Alan", "Linus") # swap one name

print(names)
print(fixed)
```

Your task

- Fill each ___ with the method that does the described job.
- Methods can be **chained**: `raw.strip().upper()` runs left to right.
- Stretch: print `len(names)` — how many names did `split()` produce?

● **Insight:** `strip()` removes spaces only at the **ends**, never in the middle.

Chained Methods

KNOWLEDGE CHECK



quiz2.py

```
text = "  Hello World  "  
print(text.strip().lower())
```

A. " hello world "

B. "HELLO WORLD"

C. "hello world"

D. "Hello World"

Python String Slicing Exercises

Exercise 1 · Basics

TRY IT YOURSELF · 3 MINUTES

```
s = "Hello, World!"
```

```
# a) Get the first 5 characters
```

```
# b) Get the last 6 characters
```

```
# c) Get everything except the first and last character
```

Your task

- s = "Hello, World!"
- Slice out the **first 5** and **last 6** characters.
- Drop the **first and last** character in one slice.

Exercise 2 · Step Slicing

TRY IT YOURSELF · 4 MINUTES

```
s = "abcdefghijklmnopqrstuvwxyz"
```

```
# a) Get every 2nd character
```

```
# b) Reverse the entire string
```

```
# c) Get every 3rd character, starting from index 1
```

Your task

- `s = "abcdefghijklmnopqrstuvwxyz"`
- Take **every 2nd character** with a step slice.
- Reverse the string using **step -1**.
- Combine a **start index and step 3**.

Exercise 3 · Negative Indices

TRY IT YOURSELF · 4 MINUTES

```
s = "Python Programming"

# a) Get the last word ("Programming") using slicing, not split()

# b) Get all characters except the last 3

# c) Get the 3rd-to-last through the last character
```

Your task

- `s = "Python Programming"`
- Pull the **last word** using negative indices, not ``split()``.
- Drop the **last 3 characters**.
- Grab only the **final 3 characters**.

Exercise 4 · Extract & Rearrange

```
date = "2026-07-12" # format: YYYY-MM-DD

# a) Extract the year, month, and day
#     separately using slicing

# b) Rearrange into DD-MM-YYYY format using
#     only slicing + string concatenation
```

Your task

- `date = "2026-07-12"`
- Slice out the **year, month, and day** separately.
- Concatenate into **DD-MM-YYYY** format.
- Use only **slicing + string ops** — no ``split()``.

Comparison & Logic

Ask true / false questions and combine them with `and` `or` `not`

True or False?

TRY IT YOURSELF · ~6 MIN

```
1  score = 72
2  pass_mark = 50
3
4  # Fill ___ so each line prints True
5
6  print(score ___ pass_mark) # score is greater
7  print(score ___ 100)      # score is not equal to 100
8  print(score ___ 72)       # score is exactly 72
9  print(score ___ pass_mark) # score is at least pass_mark
```

Your task

- Fill each ___ so that every line prints **True**.
- Choose from == != < > <= >=.
- A comparison always returns a **boolean** — True or False.

Fix the Bugs

DEBUG CHALLENGE

1

1 · Comparing an int with a str

```
bug1.py

number = 10
text = "5"
print(number > text)  # raises TypeError
```

2

2 · Assigning instead of comparing

```
bug2.py

age = 18
if age = 18:           # raises SyntaxError
    print("adult")
```

3

3 · Spell out your logic words

Fix each snippet above, then rewrite this correctly: use `age > 0` and `age < 120` — the words and / or / not, not the symbols & or |.

Logic Gates

TRY IT YOURSELF · ~8 MIN

```
entry_rules.py

age = 20
has_ticket = True
is_vip = False

# Fill ___ with and / or / not
can_enter = (age >= 18) ___ has_ticket    # BOTH needed
skip_line = is_vip ___ (age >= 65)        # EITHER is enough
needs_id = ___ is_vip                    # everyone NOT a vip

print(can_enter, skip_line, needs_id)
```

Your task

- Fill each ___ with **and**, **or**, or **not**.
- **and** needs both true; **or** needs at least one; **not** flips it.
- Predict the three booleans, then run to check.

Insight: **not** binds tightly: not is_vip flips just is_vip, not the whole line.